Datum — Documentation Index

Platform overview and service catalog · Internal engineering documentation

1. Introduction

1.1 What is Datum?

Datum is a multi-tenant e-commerce merchant dashboard platform. It gives merchants (brands) a single place to see how their store is performing — sales and product metrics, session and web-vitals analytics, inventory alerts, daily operational insights, and merchant support requests — pulled together from multiple backend services behind one authenticated gateway. The production frontend is served at `datum.trytechit.co`.

1.2 Overall Goal

The platform exists to consolidate operational visibility for multiple brands (tenants) into one product, while keeping each brand's data isolated and access-controlled. Concretely, Datum aims to:

- Give merchants and internal staff a unified dashboard for metrics, alerts, session behavior, and daily insights across brands they have access to.
- Ingest and process operational events (sessions, web vitals, inventory changes, tracked events) in near real time.
- Route and authenticate every request per-tenant, so a user only ever sees data for the brand(s) they're authorized on.
- Surface the health of the platform itself (service uptime, registered routes, health checks) through a dedicated monitoring layer.
- Keep merchant-facing operational workflows (requests/tickets, alerts, daily notes) inside the same product instead of scattered across external tools.

1.3 How It Works

Datum is a set of independently deployed Node.js services (plus one Lua/OpenResty gateway) that sit behind a single entry point, with a React frontend consuming them all through that entry point.

- **Entry point** — All external traffic goes through `api-gateway`, an OpenResty/nginx reverse proxy. It routes requests by path prefix to the correct backend service, verifies JWTs and enforces rate

limits on protected routes via embedded Lua modules, and leaves a small set of routes (public tracking/webhook endpoints) unauthenticated by design.

- **Identity** — `auth-service` (bundled with the gateway's codebase, run as its own container) owns signup/login, refresh/logout, Google OAuth, and publishes a JWKS endpoint that the gateway's Lua layer uses to verify tokens. It also manages admin-level user, domain-rule, and role administration.
- **Tenancy** — `tenant-router` is the source of truth for which brand maps to which database shard and pipeline credentials. Other services resolve tenant context through it, and it also relays brand onboarding jobs to an external pipeline orchestrator.
- **Domain services** — Independent services own their own slice of functionality: analytics/reporting (`analytics`), inventory + push alerts (`alerts-service`), merchant support tickets synced to Todoist (`merchant-requests-service`), per-brand daily notes (`daily-insights-service`), and raw session ingestion (`sessions-service`). Each is a small Express app with its own database access, deployed and health-checked independently via Docker Compose.
- **Platform health** — `health-monitor-service` is a registry that every other service reports into on startup (its health-check config and discovered routes). The dashboard's Health Monitor panel reads aggregated status from it.
- **Frontend** — `client/dashboard`, a React + Vite single-page app, is the surface merchants and staff use. It talks to every backend service exclusively through the gateway, and renders KPIs, funnels, inventory, alerts, requests, session/web-vitals analytics, and the health monitor panel.
- **Deployment** — The whole stack (gateway, auth, tenant-router, alerts, merchant-requests, daily-insights, analytics, sessions) is brought up together via `scripts/compose-stack.js` / `docker-compose.yml`. `health-monitor-service` runs from its own compose file since it isn't a hard dependency of the gateway.

---

2. Services

#	Service	Port	One-line purpose
1	api-gateway	18080 (exposed as 8081)	Edge reverse proxy: routing, JWT auth, rate limiting
2	auth-service	3001	Identity: signup/login, OAuth, JWKS, user/role admin
3	tenant-router	3004	Tenant resolution, onboarding relay, pipeline credentials

4	alerts-service	5005	Alert rules, inventory cache, event tracking, push notifications
5	merchant-requests-service	4020	Merchant support requests synced with Todoist
6	daily-insights-service	4030	Per-brand daily insight notes
7	analytics	3006	Core metrics, reporting, uploads, API keys, Shopify integration
8	sessions-service	4010	Session ingestion and de-duplication
9	health-monitor-service	4015	Central service health registry and reporting
10	client/dashboard	— (static SPA)	React frontend consumed by merchants and staff

2.1 api-gateway

Purpose: The single public entry point for the entire platform, built on OpenResty (nginx + Lua). It routes incoming requests by path prefix (`/analytics`, `/alerts`, `/merchant-requests`, `/daily-insights`, `/tenant`, `/sessions`, `/health-monitor`, plus Socket.IO passthroughs for `tenant-router` and `merchant-requests-service`) to the correct backend service. Embedded Lua modules handle JWT verification (`auth.lua`, `jwks.lua`) and rate limiting (`ratelimit.lua`) on protected routes; a small set of routes (`/track`, `/events`, `/inventory`, the Todoist webhook) are deliberately left public or gated by a shared pipeline key instead of a user JWT. It also exposes `/health` and `/nginx_status`.

2.2 auth-service

Purpose: Owns identity for the platform. Built from `api-gateway/Dockerfile.auth` and backed by MongoDB. Handles signup, login, refresh, logout, `/me`, Google OAuth (`/google/start`, `/google/callback`), and publishes a JWKS endpoint (`/.well-known/jwks.json`) that the gateway's Lua layer uses to verify JWTs. Also exposes admin endpoints for user management, domain rules, and custom roles.

2.3 tenant-router

Purpose: Central tenant registry and onboarding relay. Resolves a brand to its database shard (`/resolve`), forwards onboarding payloads to an external pipeline orchestrator and relays onboarding logs over Socket.IO (`/onboard`, `/onboard/logs`), manages CDS brand mappings and tenant CRUD (`/brands`, `/create`, `/cds/mappings`), and looks up tenants by credentials. A separate pipeline-key-

gated route group manages encrypted per-brand pipeline database credentials and validates onboarding "speed keys."

2.4 alerts-service

Purpose: Handles alert configuration, inventory caching, event tracking, and push notifications. Provides CRUD for alert rules (author-only), refreshes an inventory cache from per-brand MySQL into Redis on a timer with a pipeline-key-gated ingest endpoint, tracks events (`/track`, with a public `/events` count endpoint), and delivers/manages FCM push notifications (`/push/receive`, `/push/register-token`, `/push/notifications`).

2.5 merchant-requests-service

Purpose: Manages merchant-submitted support requests, kept in sync with Todoist. Supports request listing/creation, comments, and assignee/due-date/status updates, plus author-only configuration and provisioning of per-brand Todoist projects. A dedicated webhook route ingests Todoist callbacks to keep request state reconciled with Todoist.

2.6 daily-insights-service

Purpose: A focused service for per-brand daily insight notes. Supports fetching an insight for a given date (permission-gated), listing history (author-only), and creating/upserting insights (author-only), all scoped per-brand.

2.7 analytics

Purpose: The core metrics and reporting service, and the largest in the platform. Covers dashboard summary and metrics endpoints, product conversion and bundle analytics, session analytics and web-vitals ingestion/query, file uploads, API key issuance and management, Shopify integration, notifications, and an external-facing API router. It also contains a `pipeline.py` file kept only as a reference copy — the live ETL pipeline that populates analytics data runs in a separate repository, not here.

2.8 sessions-service

Purpose: A minimal session-ingestion endpoint. Records a user session (id, user/email, brand, device/user-agent, IP, route) on `POST /sessions`, enforcing session ID uniqueness and de-duplicating repeat sessions from the same user within a rolling window (`SESSION_WINDOW_MINUTES`, default 30). Brand and role context are taken from gateway-injected headers when present.

2.9 health-monitor-service

Purpose: The platform's central health/registry hub. Every other service self-registers its health-check configuration and discovered routes with it on startup. It exposes `/register`, `/events` for application event ingestion, and a permission-gated query API that the dashboard's Health Monitor panel uses to read aggregated service status. It is deployed via its own compose file since it is not a hard startup dependency of the gateway.

2.10 client/dashboard

Purpose: The Datum web dashboard — a React + Vite single-page application, served in production at `datum.trytechit.co`. It provides the merchant/staff-facing UI: KPI tiles, daily funnel and insights panels, inventory tables, alerts administration, the merchant-requests panel, author/brand management, notifications, session analytics and web-vitals views, mobile navigation, and the Health Monitor panel. All data access goes through `api-gateway`.